

Comprehensive Notes on JDBC and Java Multithreading

PART 1: JDBC (Java Database Connectivity)

1. Introduction to JDBC

JDBC (Java Database Connectivity) is a Java API that enables Java applications to interact with databases. It provides methods for querying and updating data in databases, regardless of the database management system being used.

2. Steps Involved in Writing a JDBC Program

1. **Import JDBC packages:** Include the necessary JDBC classes and interfaces using `import java.sql.*;`.
2. **Load and register the JDBC driver:** Before connecting to a database, load the appropriate driver.

```
java
```

```
Class.forName("com.mysql.cj.jdbc.Driver"); // for MySQL 8.0+
```

3. **Establish a connection:** Create a connection to the database using the DriverManager class.

```
java
```

```
String url = "jdbc:mysql://localhost:3306/dbname";  
String username = "root";  
String password = "password";  
Connection conn = DriverManager.getConnection(url, username, password);
```

4. **Create a statement object:** Create a Statement, PreparedStatement, or CallableStatement object.

```
java
```

```
Statement stmt = conn.createStatement();  
// or  
PreparedStatement pstmt = conn.prepareStatement("INSERT INTO users VALUES(?, ?)");
```

5. **Execute SQL queries:** Use the statement object to execute SQL commands.

```
java
```

```
ResultSet rs = stmt.executeQuery("SELECT * FROM users");  
// or  
int rowsAffected = stmt.executeUpdate("UPDATE users SET name='John' WHERE id=1");
```

6. **Process the results:** Process the data retrieved from the database.

```
java
```

```
while(rs.next()) {  
    System.out.println(rs.getInt("id") + " " + rs.getString("name"));  
}
```

7. **Close the connection:** Release the resources.

```
java
```

```
rs.close();  
stmt.close();  
conn.close();
```

3. Statement Execution Methods in JDBC

Differentiation between execute(), executeUpdate(), and executeQuery()

Method	Purpose	Return Type	Use Case
executeQuery()	Used for SELECT queries that retrieve data	Returns a ResultSet object	For all SELECT queries
executeUpdate()	Used for INSERT, UPDATE, DELETE, and DDL statements	Returns an int representing the number of rows affected	For data manipulation and definition operations
execute()	Used when the query could be any type of SQL statement	Returns a boolean (true if result is a ResultSet, false otherwise)	When the type of SQL statement is not known in advance

Code examples:

java

```
// executeQuery - returns ResultSet
ResultSet rs = stmt.executeQuery("SELECT * FROM employees");

// executeUpdate - returns int (number of rows affected)
int rowsUpdated = stmt.executeUpdate("UPDATE employees SET salary = 50000 WHERE id = 101");

// execute - returns boolean
boolean hasResultSet = stmt.execute("SELECT * FROM employees");
if (hasResultSet) {
    ... ResultSet rs = stmt.getResultSet();
    ... // process ResultSet
} else {
    ... int updateCount = stmt.getUpdateCount();
    ... // process update count
}
```

4. Types of JDBC Drivers

JDBC drivers are classified into four types:

Type 1: JDBC-ODBC Bridge Driver

Architecture:

Java Application → JDBC API → JDBC-ODBC Bridge → ODBC Driver → Database

Pros:

- Easy to use for Windows applications
- Can access any database that has an ODBC driver

Cons:

- Performance overhead due to multiple layers
- Requires ODBC installation on the client machine
- Platform-dependent (since ODBC is generally available on Windows)
- Deprecated since Java 8

Type 2: Native-API Driver (Partially Java Driver)

Architecture:

Java Application → JDBC API → Native API Driver → Client Database Library → Database

Pros:

- Better performance than Type 1 drivers
- Uses database vendor's client libraries

Cons:

- Requires installation of native client libraries on the client machine
- Platform-dependent
- Database-specific

Type 3: Network Protocol Driver (Pure Java Driver)

Architecture:

Java Application → JDBC API → JDBC-Net Pure Java Driver → Middleware → Database

Pros:

- Platform-independent (100% pure Java)
- No need for client-side libraries
- Can connect to multiple database types through middleware
- Good for web applications

Cons:

- Requires middleware/application server configuration
- Additional network overhead
- Middleware becomes a potential single point of failure

Type 4: Thin Driver (Pure Java Driver)

Architecture:

Java Application → JDBC API → Pure Java Driver → Database

Pros:

- Platform-independent (100% pure Java)
- Best performance among all driver types
- Direct connection to the database
- No need for additional software layers
- Most commonly used driver type today

Cons:

- Database-specific
- Different drivers needed for different databases

5. Sample JDBC Program

Here's a complete program that demonstrates both selecting and updating records:

java

```

import java.sql.*;

public class JDBCExample {
    .... // Database connection parameters
    .... static final String DB_URL = "jdbc:mysql://localhost:3306/employeedb";
    .... static final String USER = "root";
    .... static final String PASS = "password";
    ....
    .... public static void main(String[] args) {
        .... Connection conn = null;
        .... Statement stmt = null;
        .... PreparedStatement pstmt = null;

        .... try {
            .... // Step 1: Register JDBC driver
            .... Class.forName("com.mysql.cj.jdbc.Driver");

            .... // Step 2: Open a connection
            .... System.out.println("Connecting to database...");
            .... conn = DriverManager.getConnection(DB_URL, USER, PASS);

            .... // Step 3: Create statement object
            .... stmt = conn.createStatement();

            .... // Step 4a: Execute a SELECT query and process results
            .... System.out.println("Retrieving data...");
            .... String selectSQL = "SELECT id, name, salary FROM employees";
            .... ResultSet rs = stmt.executeQuery(selectSQL);

            .... // Process the result set
            .... System.out.println("Current Employee Data:");
            .... System.out.println("ID\tName\t\tSalary");
            .... while(rs.next()) {
            ....     int id = rs.getInt("id");
            ....     String name = rs.getString("name");
            ....     double salary = rs.getDouble("salary");
            ....     System.out.println(id + "\t" + name + "\t\t" + salary);
            .... }

            .... // Step 4b: Execute an UPDATE query using PreparedStatement
            .... System.out.println("\nUpdating employee salaries...");
            .... String updateSQL = "UPDATE employees SET salary = ? WHERE id = ?";
            .... pstmt = conn.prepareStatement(updateSQL);

```

```

.....
        // Set the parameters
..... pstmt.setDouble(1, 55000.00); // New salary
..... pstmt.setInt(2, 101); ..... // Employee ID

..... // Execute the update
..... int rowsAffected = pstmt.executeUpdate();
..... System.out.println(rowsAffected + " row(s) updated");

.....
..... // Step 4c: Verify the update with another SELECT query
..... rs = stmt.executeQuery(selectSQL);
.....
..... // Process the result set again
..... System.out.println("\nUpdated Employee Data:");
..... System.out.println("ID\tName\t\tSalary");
..... while(rs.next()) {
.....     int id = rs.getInt("id");
.....     String name = rs.getString("name");
.....     double salary = rs.getDouble("salary");
.....     System.out.println(id + "\t" + name + "\t\t" + salary);
..... }

.....
..... // Step 5: Clean up resources
..... rs.close();
..... stmt.close();
..... pstmt.close();
..... conn.close();

.....
..... } catch(SQLException se) {
.....     // Handle errors for JDBC
.....     se.printStackTrace();
..... } catch(Exception e) {
.....     // Handle errors for Class.forName
.....     e.printStackTrace();
..... } finally {
.....     // Finally block to close resources
.....     try {
.....         if(stmt != null) stmt.close();
.....     } catch(SQLException se2) { }
.....     try {
.....         if(pstmt != null) pstmt.close();
.....     } catch(SQLException se2) { }
.....     try {
.....         if(conn != null) conn.close();

```

```

        } catch(SQLException se) {
            se.printStackTrace();
        }
    }
    }
    System.out.println("Program completed.");
}
}
}

```

This program assumes there's a MySQL database named "employeeedb" with a table called "employees" containing columns for id, name, and salary. It demonstrates:

- Connection establishment
- Creating Statement and PreparedStatement objects
- Executing SELECT queries with executeQuery()
- Executing UPDATE statements with executeUpdate()
- Processing ResultSet data
- Proper resource management with try-catch-finally blocks

PART 2: Threading & Concurrency in Java

1. Introduction to Threads

A thread is the smallest unit of execution within a process. Java is a multi-threaded programming language, which means it can run multiple threads concurrently.

2. Thread vs Process

Feature	Process	Thread
Definition	Independent program in execution	Lightweight sub-process, a unit of execution within a process
Resource Ownership	Has its own memory space	Shares memory with other threads in the same process
Communication	Inter-process communication is complex and resource-intensive	Thread communication is simpler as they share memory
Context Switching	More time-consuming	Faster and less resource-intensive
Creation	More system resources required	Less system resources required
Failure Impact	Other processes are not affected if one fails	If one thread crashes, it may affect other threads in the process

3. Thread Lifecycle

The lifecycle of a thread in Java includes the following states:

1. **New:** A thread is in this state when it has been created but not yet started.
2. **Runnable:** After calling the `start()` method, the thread enters the runnable state. It's ready to run but may not be actively executing.
3. **Running:** The thread is actively executing when the JVM thread scheduler selects it.
4. **Blocked/Waiting:** The thread is temporarily inactive, waiting for a resource, I/O operation, or for another thread to complete a task.
5. **Timed Waiting:** Similar to waiting, but with a specified waiting time.
6. **Terminated:** The thread has completed its execution or was stopped.

Thread Lifecycle Diagram:



4. Ways to Create Threads in Java

There are two primary ways to create threads in Java:

A. Extending the Thread Class

java

```
class MyThread extends Thread {  
    public void run() {  
        ..... System.out.println("Thread is running using Thread class extension");  
        ..... }  
  
    ..... public static void main(String args[]) {  
        ..... MyThread t1 = new MyThread();  
        ..... t1.start(); // starts the thread  
        ..... }  
    }  
}
```

B. Implementing the Runnable Interface

java

```
class MyRunnable implements Runnable {  
    ..... public void run() {  
        ..... System.out.println("Thread is running using Runnable interface");  
        ..... }  
    .....  
    ..... public static void main(String args[]) {  
        ..... MyRunnable myRunnable = new MyRunnable();  
        ..... Thread t1 = new Thread(myRunnable);  
        ..... t1.start(); // starts the thread  
        ..... }  
    }  
}
```

C. Using Lambda Expressions (Java 8+)

java

```
public class ThreadWithLambda {
    public static void main(String[] args) {
        // Using Runnable with Lambda
        Runnable r = () -> {
            System.out.println("Thread is running using lambda expression");
        };

        Thread t = new Thread(r);
        t.start();

        // Shorter version
        new Thread(() -> System.out.println("Thread with inline lambda")).start();
    }
}
```

D. Using the ExecutorService (Java 5+)

java

```
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class ThreadWithExecutorService {
    public static void main(String[] args) {
        ExecutorService executor = Executors.newSingleThreadExecutor();

        executor.submit(() -> {
            System.out.println("Thread is running using ExecutorService");
        });

        executor.shutdown();
    }
}
```

5. Thread Synchronization

Thread synchronization is a mechanism that ensures that only one thread can access a shared resource at a time to prevent data inconsistency.

Why Synchronization is Needed:

When multiple threads try to access and modify shared resources simultaneously, it can lead to race conditions and data corruption. Synchronization helps maintain data consistency.

Methods of Thread Synchronization:

1. Synchronized Methods:

```
java

public synchronized void synchronizedMethod() {
    .... // Only one thread can execute this method at a time
    .... // for the same object instance
}
```

2. Synchronized Blocks:

```
java

public void method() {
    .... // Non-synchronized code can be executed by multiple threads
    ....
    .... synchronized(this) { // 'this' is the lock object
        .... // Only one thread can execute this block at a time
        .... // for the same lock object
    }
    ....
    .... // More non-synchronized code
}
```

3. Static Synchronization:

```
java

public static synchronized void staticSynchronizedMethod() {
    .... // Only one thread can execute this method at a time
    .... // across all instances of the class
}
```

4. Lock Interface (java.util.concurrent.locks):

java

```
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;

class LockExample {
    private final Lock lock = new ReentrantLock();

    ....
    public void method() {
        lock.lock(); // Acquire the Lock
        try {
            .... // Critical section - protected by the Lock
        } finally {
            lock.unlock(); // Always release the lock in a finally block
        }
    }
}
```

5. Volatile Keyword:

java

```
private volatile boolean flag = false;
```

The `volatile` keyword ensures that changes to a variable are always visible to other threads. It doesn't provide mutual exclusion but helps with visibility issues.

6. Thread Communication

Threads often need to communicate with each other, especially in producer-consumer scenarios.

Methods for Thread Communication:

1. **wait()**: Causes the current thread to wait until another thread invokes `notify()` or `notifyAll()`
2. **notify()**: Wakes up a single thread that is waiting on the monitor
3. **notifyAll()**: Wakes up all threads that are waiting on the monitor

java

```
class SharedResource {
    private boolean dataProduced = false;
    private int data;

    public synchronized void produce(int value) throws InterruptedException {
        while (dataProduced) {
            // Wait if data is already produced but not consumed
            wait();
        }

        // Produce data
        data = value;
        dataProduced = true;
        System.out.println("Produced: " + data);

        // Notify consumer
        notify();
    }

    public synchronized int consume() throws InterruptedException {
        while (!dataProduced) {
            // Wait if no data is produced yet
            wait();
        }

        // Consume data
        dataProduced = false;
        System.out.println("Consumed: " + data);

        // Notify producer
        notify();

        return data;
    }
}
```

7. Common Thread Methods

Method	Description
<code>start()</code>	Starts the thread execution
<code>run()</code>	Contains the code that will be executed in the thread
<code>sleep(long millis)</code>	Makes the thread pause execution for a specified period
<code>join()</code>	Waits for this thread to die
<code>yield()</code>	Causes the current thread to pause temporarily and allow other threads to execute
<code>interrupt()</code>	Interrupts a thread in sleep or wait state
<code>isAlive()</code>	Tests if the thread is alive
<code>setPriority(int)</code>	Sets thread priority (1-10, with 10 being highest)

8. Multithreading Concepts in Java

Thread Pools

Thread pools manage a collection of worker threads, which helps to improve application performance by reusing existing threads rather than creating new ones.

```
java
```

```
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class ThreadPoolExample {
    public static void main(String[] args) {
        ..... // Create a fixed size thread pool with 5 threads
        ..... ExecutorService executor = Executors.newFixedThreadPool(5);

        ..... // Submit tasks for execution
        ..... for (int i = 0; i < 10; i++) {
            ..... final int taskId = i;
            ..... executor.execute(() -> {
                ..... System.out.println("Task " + taskId + " executed by " +
                ..... Thread.currentThread().getName());
            ..... });
        ..... }

        ..... // Shutdown the executor
        ..... executor.shutdown();
    ..... }
}
```

Deadlock

Deadlock occurs when two or more threads are blocked forever, each waiting for the other to release a lock.

java

```
public class DeadlockExample {
    .... public static void main(String[] args) {
        .... final Object resource1 = "resource1";
        .... final Object resource2 = "resource2";
        ....
        // Thread 1
        .... Thread t1 = new Thread(() -> {
            .... synchronized (resource1) {
                .... System.out.println("Thread 1: locked resource 1");
                ....
                .... try { Thread.sleep(100); } catch (Exception e) {}
                ....
                .... synchronized (resource2) {
                    .... System.out.println("Thread 1: locked resource 2");
                    .... }
                .... }
            .... });
        ....
        // Thread 2
        .... Thread t2 = new Thread(() -> {
            .... synchronized (resource2) {
                .... System.out.println("Thread 2: locked resource 2");
                ....
                .... try { Thread.sleep(100); } catch (Exception e) {}
                ....
                .... synchronized (resource1) {
                    .... System.out.println("Thread 2: locked resource 1");
                    .... }
                .... }
            .... });
        .... t1.start();
        .... t2.start();
        .... }
    }
}
```

Race Conditions

Race conditions occur when two or more threads access shared data and try to change it at the same time.

java

```
public class RaceConditionExample {
    private static int counter = 0;

    ....
    public static void main(String[] args) throws InterruptedException {
        // Without synchronization - Race condition
        .... Thread t1 = new Thread(() -> {
            .... for (int i = 0; i < 1000; i++) {
                counter++;
            }
        });

        .... Thread t2 = new Thread(() -> {
            .... for (int i = 0; i < 1000; i++) {
                counter++;
            }
        });

        .... t1.start();
        .... t2.start();

        .... t1.join();
        .... t2.join();

        .... System.out.println("Final counter value: " + counter);
        .... // Expected: 2000, but due to race condition, it might be less
    }
}
```

Thread-Safe Collections

The `java.util.concurrent` package provides thread-safe collection classes designed for concurrent access.

java

```
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;

public class ThreadSafeCollectionsExample {
    public static void main(String[] args) {
        ..... // Thread-safe map
        ..... Map<String, String> concurrentMap = new ConcurrentHashMap<>();

        ..... // Can be safely used by multiple threads
        ..... concurrentMap.put("key1", "value1");
        ..... concurrentMap.put("key2", "value2");

        .....
        ..... System.out.println(concurrentMap.get("key1"));
        ..... }
    }
}
```

9. Complete Thread Example

Here's a comprehensive example that demonstrates thread creation, synchronization, and communication:

java


```

class Counter {
.... private int count = 0;
....
.... // Synchronized method
.... public synchronized void increment() {
....     count++;
.... }
....
.... public synchronized int getCount() {
....     return count;
.... }
}

class WorkerThread extends Thread {
.... private Counter counter;
.... private int incrementTimes;
....
.... public WorkerThread(Counter counter, int incrementTimes, String name) {
....     super(name);
....     this.counter = counter;
....     this.incrementTimes = incrementTimes;
.... }

.... @Override
.... public void run() {
....     for (int i = 0; i < incrementTimes; i++) {
....         counter.increment();
....         System.out.println(getName() + " incremented counter to: " + counter.getCount());

....         try {
....             // Random sleep to simulate work
....             Thread.sleep((long) (Math.random() * 50));
....         } catch (InterruptedException e) {
....             System.out.println(getName() + " was interrupted");
....             return;
....         }
....     }
....     System.out.println(getName() + " finished execution");
.... }
}

public class ThreadingDemo {
.... public static void main(String[] args) throws InterruptedException {

```

```

..... System.out.println("Main thread starting");

..... // Shared resource
..... Counter sharedCounter = new Counter();

..... // Create worker threads
..... WorkerThread thread1 = new WorkerThread(sharedCounter, 5, "Thread-1");
..... WorkerThread thread2 = new WorkerThread(sharedCounter, 5, "Thread-2");

.....
..... // Start the threads
..... thread1.start();
..... thread2.start();

.....
..... // Main thread waits for worker threads to complete
..... thread1.join();
..... thread2.join();

.....
..... System.out.println("All threads have finished");
..... System.out.println("Final counter value: " + sharedCounter.getCount());
..... System.out.println("Main thread exiting");
..... }
}

```

This example shows:

- Thread creation by extending Thread class
- Synchronization using synchronized methods
- Thread communication through a shared Counter object
- Thread coordination using join()
- Thread states via sleep() and execution cycles

Summary

JDBC Summary

- JDBC provides a standard API for connecting Java applications to databases
- The main steps in a JDBC program are: loading the driver, establishing a connection, creating statements, executing queries, processing results, and closing resources
- The three main execute methods are executeQuery() (for SELECT), executeUpdate() (for INSERT/UPDATE/DELETE), and execute() (when the query type is unknown)

- JDBC drivers come in four types, with Type 4 (Pure Java) being the most commonly used today

Threading Summary

- Threads allow concurrent execution within a Java program
- Threads can be created by extending Thread class or implementing Runnable interface
- Thread synchronization mechanisms prevent race conditions
- Thread lifecycle includes: New, Runnable, Running, Waiting/Blocked, and Terminated states
- Thread communication can be facilitated using wait(), notify(), and notifyAll() methods

Both JDBC and threading are fundamental concepts in Java programming that enable database connectivity and concurrent program execution, respectively. Understanding these concepts is crucial for developing efficient and robust Java applications.